WEEK 4 · SECTION 4

Schema Enforcement & Evolution

Official Databricks Documentation Basis | May 2026 Exam Guide

- ✥ Schema inference fundamentals
- ✥ Schema evolution modes (official names)
- ✥ Rescued data column deep dive
- ✥ Type widening & schema hints
- ✥ DLT integration patterns
- ✥ 15 practice questions

Table of Contents

1. Schema Fundamentals 3

2. Schema Inference Deep Dive 6

3. Schema Evolution Modes (Corrected) 10

4. The Rescued Data Column 15

5. Schema Hints & Type Enforcement 19

6. Integration with DLT & writeStream 23

7. Best Practices & Production Patterns 27

8. Week 3 + Week 4 Complete Framework 31

9. Practice Questions (15 Qs) 34

10. Deep Dive: Type Widening Lab 40

11. Answer Key 43

12. Master Cheat Sheet 46

SECT 1

1. Schema Fundamentals

Auto Loader Inference & Enforcement

1.1 What Is Schema In Auto Loader?

A **schema** is the structural blueprint of your data—the column names, data types, and nested structures. In Auto Loader, you can either:

- **Infer the schema** automatically by sampling source files
- **Provide a schema** manually as a StructType or schema hints
- **Evolve the schema** as new columns arrive in your source data

1.2 When Does Schema Inference Occur?

■ **Auto Loader First-Run Inference**

```
# On first run, Auto Loader samples up to:
# • 50 GB of data, OR
# • 1000 files, whichever limit is reached first

df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaLocation', 's3://bucket/schema')
  # ^ TRIGGERS schema inference + storage
  .load('s3://bucket/data/')
```

> ■ **Note**
>
> On **first run**, Auto Loader creates a _schemas directory inside cloudFiles.schemaLocation to track schema evolution over time.

1.3 Supported File Formats For Schema Inference

Format	Supported?	Default Type Inference	Notes
JSON	✓ Yes	STRING (all columns)	Use cloudFiles.inferColumnTypes=true
CSV	✓ Yes	STRING (all columns)	Assumes headers present
XML	✓ Yes (DBR 14.3+)	STRING	New feature, requires recent runtime
Avro	✓ Yes (DBR 10.4+)	From Avro schema	Type-aware by default
Parquet	✓ Yes (DBR 11.3+)	From Parquet schema	Uses widest type if mismatch
ORC	✗ No	N/A	Not supported for inference
Text	✗ No	Fixed STRING	No inference needed
binaryFile	✗ No	Fixed BINARY	No inference needed

> ■■ **Warning**

ORC is NOT supported for schema inference in Auto Loader, despite being a popular columnar format. Use Parquet instead for schema-evolving ingestion.

SECT 2

2. Schema Inference Deep Dive

Sampling, Type Inference, Merging

2.1 Default Inference Behavior by Format

For JSON, CSV, XML (Untyped Formats):

By default, **all columns are inferred as STRING**. To enable actual type inference (INT, BOOLEAN, TIMESTAMP), set cloudFiles.inferColumnTypes=true.

■ Type Inference Toggle

```
# Default: all STRING
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')
# Result: event_id STRING, timestamp STRING, amount STRING

# Enable type inference
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.inferColumnTypes', 'true')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')
# Result: event_id STRING, timestamp TIMESTAMP, amount DOUBLE
```

For Parquet & Avro (Typed Formats):

Schema is read from the file metadata. **Auto Loader samples files and merges schemas** to handle evolution. When columns have different types across files, **the widest type is chosen** (e.g., INT → BIGINT → DOUBLE).

2.2 Schema Sampling Configuration

Auto Loader samples data to infer schema. By default it uses the first 50 GB or 1000 files. You can customize this via Spark SQL configurations:

■ Customizing Sample Size

```
# Set sample size in bytes
spark.conf.set('spark.databricks.cloudFiles.schemaInference.sampleSize.numBytes', '10gb')

# Set sample size in number of files
spark.conf.set('spark.databricks.cloudFiles.schemaInference.sampleSize.numFiles', '500')
```

■ Tip

Larger samples = more accurate schema but longer first run. Balance accuracy vs. startup time based on your data characteristics.

2.3 Case Sensitivity in Schema Inference

Auto Loader treats abc, Abc, and ABC as the **same column** unless case sensitivity is enabled. It **arbitrarily chooses** one case based on sampled data. Use cloudFiles.schemaHints or readerCaseSensitive=false to control behavior.

■ Case Sensitivity Example

```
# If source has: event_id, Event_ID, EVENT_ID
# Auto Loader picks one (e.g., event_id) and treats others as duplicates
```

```
# Use schemaHints to enforce case
.option('cloudFiles.schemaHints', 'event_id INT, timestamp TIMESTAMP')
```

SECT 3

3. Schema Evolution Modes — Corrected

Official cloudFiles.schemaEvolutionMode Options

3.1 Overview of All Evolution Modes

When Auto Loader detects **new columns** in source data, it triggers schema evolution. The cloudFiles.schemaEvolutionMode option controls how Auto Loader responds. There are **5 official modes**:

Mode Name	Behavior	Stream State	Restart Required?
addNewColumns (default)	Adds new columns to schema, stream fails with UnknownFieldException	FAILS after detection	YES — manual
addNewColumnsWithTypeWidening	Like addNewColumns + auto-widens types (int→long). DBR 16.4+	FAILS after detection	YES — manual
rescue	Schema unchanged. New columns saved to _rescued_data JSON col	CONTINUES normally	NO
failOnNewColumns	Stream fails, will not auto-update. Requires manual schema fix	FAILS and STAYS failed	YES — manual schema
none	New columns ignored entirely (not rescued)	CONTINUES normally	NO

■ Exam Focus

Default mode **depends on schema source**:
• No schema provided → **addNewColumns** (default)
• Schema provided explicitly → **none** (default)

3.2 addNewColumns — The Default Mode

■ addNewColumns (Default) Flow

```
# When Auto Loader encounters a NEW column:
# 1. It performs schema inference on latest micro-batch
# 2. Adds new columns to the schema in cloudFiles.schemaLocation
# 3. Stream STOPS with UnknownFieldException
# 4. User restarts stream → schema is updated

# Default behavior (no explicit option needed)
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaLocation', '/schema')
  # ^ Implicitly uses addNewColumns mode
  .load('/data/')
```

3.3 rescue — Never Evolve, Always Capture

■ Using rescue Mode

```
# In rescue mode, stream NEVER FAILS due to schema changes
```

```
# New columns go to _rescued_data (JSON blob)

df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaEvolutionMode', 'rescue')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')

(df.writeStream
  .format('delta')
  .option('checkpointLocation', '/ckpt')
  .table('bronze.events'))
```

When new column region arrives, it's captured in _rescued_data. Later, you can inspect and migrate it to the main schema manually.

■ Querying Rescued Data

```
-- Inspect rescued data
SELECT event_id, _rescued_data FROM bronze.events
WHERE _rescued_data IS NOT NULL LIMIT 5;

-- Extract field from rescued data
SELECT
  event_id,
  get_json_object(_rescued_data, '$.region') AS region_from_rescue
FROM bronze.events
WHERE _rescued_data IS NOT NULL;
```

3.4 addNewColumnsWithTypeWidening — Auto Type Evolution

New in **Databricks Runtime 16.4+**. Automatically widens data types without requiring manual schema updates. For example, INT → BIGINT, FLOAT → DOUBLE.

■ Type Widening in Action

```
# Enable type widening
.option('cloudFiles.schemaEvolutionMode', 'addNewColumnsWithTypeWidening')

# Example: First file has amount INT
# Later file has amount BIGINT
# Auto Loader widens to BIGINT automatically (no stream failure)

# Unsupported widening (e.g., INT to STRING)
# → placed in _rescued_data column
```

3.5 failOnNewColumns — Strict Schema Enforcement

Stream fails immediately upon detecting a new column and **will NOT auto-update** the schema. Requires manual intervention: update schema or remove offending files.

■ Strict Mode: failOnNewColumns

```
.option('cloudFiles.schemaEvolutionMode', 'failOnNewColumns')

# When new column detected:
# • Stream fails with error
# • Schema NOT auto-updated
# • Manual fix required (update schema or delete bad files)
```

3.6 none — Ignore New Columns Silently

New columns are silently ignored (dropped). No rescue, no error. Useful when you have a fixed schema and don't care about unexpected columns.

■ Silent Ignore: none Mode

```
.option('cloudFiles.schemaEvolutionMode', 'none')

# New columns are silently dropped
# No stream failure, no rescue column populated
```

> **■ Exam Focus**
>
> The default mode **changes based on whether you provide a schema**:
> • **Schema NOT provided** → defaults to addNewColumns
> • **Schema provided** → defaults to none

3.7 Comparison Table: When To Use Each Mode

Scenario	Recommended Mode	Why?
First-time ingestion, unknown schema	addNewColumns	Auto-adapt as data arrives
Schema mostly fixed, but new fields OK	rescue	Capture surprises without failing

Strict schema contract (financial data)	failOnNewColumns	Enforce schema discipline
Schema known at start, strict	none	Explicitly provided schema; ignore extras
Need automatic type widening	addNewColumnsWithTypeWidening	Reduce manual intervention (DBR 16.4+)

SECT 4

4. The Rescued Data Column

_rescued_data & rescuedDataColumn Option

4.1 What Is the Rescued Data Column?

When Auto Loader **infers a schema**, it automatically adds a column called **_rescued_data** (by default) to capture data that doesn't fit the schema. This includes:

- New columns not in the schema
- Type mismatches (value can't be cast to declared type)
- Case mismatches (column names differ only in case)

The _rescued_data column contains a **JSON blob** with the rescued fields plus the source file path. It is NULL when no data needs rescue.

4.2 Example: _rescued_data in Action

Initial schema (from first 1000 files):

■ Inferred Schema (Day 1)

```
event_id STRING
event_ts TIMESTAMP
user_id STRING
_rescued_data STRING
```

Day 5: New field 'region' arrives in source files

■ Source Data with New Field

```
-- Source JSON now includes:
{
  'event_id': 'evt_123',
  'event_ts': '2026-08-17T10:00:00Z',
  'user_id': 'usr_456',
  'region': 'us-west-2'
}
```

Table after processing:

■ Result with Rescued Data

```
event_id  | event_ts              | user_id | _rescued_data
----------|-----------------------|---------|-----------------------
evt_123   | 2026-08-17 10:00:00  | usr_456 | {"region":"us-west-2",..}
```

4.3 Querying Rescued Data

■ Querying Rescued Data (SQL)

```
-- Find records with rescued data
SELECT event_id, _rescued_data
FROM bronze.events
WHERE _rescued_data IS NOT NULL;

-- Extract a specific field from rescued data
SELECT
  event_id,
  get_json_object(_rescued_data, '$.region') AS region
FROM bronze.events
```

```
WHERE _rescued_data IS NOT NULL;

-- Use from_json for structured access
SELECT
  event_id,
  from_json(_rescued_data, 'region STRING, new_field INT') AS rescued
FROM bronze.events
WHERE _rescued_data IS NOT NULL;
```

4.4 Renaming the Rescued Data Column (rescuedDataColumn Option)

By default, rescued data goes to _rescued_data. Use the **rescuedDataColumn** option to rename it:

■ Using rescuedDataColumn Option

```
# Rename rescued data column
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('rescuedDataColumn', 'malformed_records')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')

# Now unexpected data goes to 'malformed_records' instead
```

> **■ Note**
>
> The rescuedDataColumn option is **NOT a cloudFiles option**—it's a standard Spark format option that works with JSON, CSV, XML, Parquet, and Avro readers.

SECT 5

5. Schema Hints & Type Enforcement

Override Inference, Enforce Types

5.1 What Are Schema Hints?

Schema hints let you override auto-inferred types for specific columns without providing a full explicit schema. Use them when you know certain columns should be a different type than what Auto Loader inferred.

■ **Using Schema Hints**

```
# Auto-inferred (all STRING):
# amount STRING (but we know it's numeric)
# dob STRING (but we want DATE)

.option('cloudFiles.schemaHints',
  'amount DOUBLE, dob DATE')
```

5.2 Schema Hints With inferColumnTypes

You can use both cloudFiles.inferColumnTypes AND cloudFiles.schemaHints together. Hints act as overrides for inferred types.

■ **Hints + inferColumnTypes**

```
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.inferColumnTypes', 'true')
  .option('cloudFiles.schemaHints',
    'amount DECIMAL(10,2), created_ts TIMESTAMP')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')

# Infer most columns, but override amount and created_ts types
```

5.3 Schema Hints For Nested Structures

For nested data (JSON objects, structs), you can hint nested fields using dot notation:

■ **Nested Field Hints**

```
# Source JSON:
{
  'user_info': {
    'id': '123',
    'dob': '1990-01-15'
  }
}

.option('cloudFiles.schemaHints',
  'user_info.id INT, user_info.dob DATE')
```

5.4 Adding Columns Via Schema Hints

If a column doesn't exist in the sampled data, you can still add it via hints. This is useful for columns that appear sporadically.

■ **Adding Missing Columns**

```
# Sampled files don't have 'region', but we know it exists
.option('cloudFiles.schemaHints', 'region STRING')
```

■ Tip

Schema hints are **NOT required** to be set. Use them selectively for columns where inference doesn't match your expectations or for adding columns not yet seen.

SECT 6

6. DLT Integration & writeStream Options

Schema Location, mergeSchema, Checkpoint

6.1 Schema Management in DLT

In **Delta Live Tables**, checkpoint and schema management are **fully automated**. You only need to specify cloudFiles.schemaLocation when using inference.

■ **DLT Bronze Table with Auto Loader**

```
import dlt

@dlt.table(
  name='bronze_events',
  comment='Raw events via Auto Loader'
)
def bronze_events():
  return (spark.readStream.format('cloudFiles')
    .option('cloudFiles.format', 'json')
    .option('cloudFiles.schemaLocation',
      '/Volumes/main/bronze/schema/events')
    .option('cloudFiles.inferColumnTypes', 'true')
    .load('s3://bucket/events/')
  )
```

✓ DLT manages checkpointLocation automatically
✓ You only specify cloudFiles.schemaLocation

6.2 The mergeSchema Option (writeStream, NOT schemaEvolutionMode)

IMPORTANT CLARIFICATION: mergeSchema is a **writeStream option**, NOT a cloudFiles.schemaEvolutionMode value. It tells Delta Lake to merge new source columns into the target table schema.

■ **mergeSchema Option (writeStream)**

```
# mergeSchema is a writeStream OPTION
(df.writeStream
  .format('delta')
  .option('mergeSchema', 'true')
  .option('checkpointLocation', '/ckpt')
  .table('bronze.events'))

# It is NOT a cloudFiles.schemaEvolutionMode value
```

■■ **Warning**

Common confusion:
■ .option('cloudFiles.schemaEvolutionMode', 'merge') (doesn't exist)
■ .option('mergeSchema', 'true') (writeStream option)

6.3 When To Use mergeSchema=true

Use mergeSchema=true when you want **Auto Loader schema evolution to automatically update the target Delta table**. Without it, new columns from Auto Loader won't be added to the table.

■ **mergeSchema=true Flow**

```
# Schema evolves, and we want Delta to evolve with it
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')

(df.writeStream
  .format('delta')
  .option('mergeSchema', 'true')
  .option('checkpointLocation', '/ckpt')
  .table('bronze.events'))
```

> ■ **Tip**
>
> mergeSchema=true + rescue mode = **safe combo** for production: stream never fails, all data captured, new columns integrated over time.

SECT 7

7. Best Practices & Production Patterns

Reliability, Performance, Governance

7.1 Production Pattern: Rescue + Restart

For most production pipelines, use this pattern:

■ **Production Pattern**

```
# Step 1: Read with rescue mode (never fails)
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaEvolutionMode', 'rescue')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')

# Step 2: Write with mergeSchema (evolve table)
(df.writeStream
  .format('delta')
  .option('mergeSchema', 'true')
  .option('checkpointLocation', '/ckpt')
  .table('bronze.events'))

# Step 3: Set up Lakeflow Job to restart automatically on schema changes
```

Benefits:

✓ Stream never fails due to schema changes
✓ All data captured (in _rescued_data if needed)
✓ Manual review of changes via rescued data
✓ Gradual schema evolution, no data loss

7.2 Schema Location Best Practices

Best Practice	Reason
Use cloud storage (S3, ADLS), not DBFS	DBFS deprecated; cloud storage more reliable
Store in same location as checkpoint	Easier to manage and back up together
One schemaLocation per stream	Prevents schema conflicts between streams
Never delete schemaLocation while active	Forces re-inference from scratch if deleted
Use volume paths when possible	Optimizes file discovery in file events mode

7.3 Monitoring Schema Changes

■ **Monitoring Rescued Data (SQL)**

```
-- Check for rescued data regularly
SELECT COUNT(*) as records_with_rescue FROM bronze.events
WHERE _rescued_data IS NOT NULL;

-- List all unique rescued fields
```

```
SELECT
  transform(
    map_keys(from_json(_rescued_data, 'map<string,string>')),
    x -> x
  ) as rescued_fields
FROM bronze.events
WHERE _rescued_data IS NOT NULL
GROUP BY rescued_fields;
```

> ■ **Tip**
>
> Set up **data quality checks** in DLT pipelines to alert when _rescued_data IS NOT NULL exceeds a threshold.

SECT 8

8. Week 3 + Week 4 Complete Framework

COPY INTO vs. Auto Loader Schema Decisions

8.1 Decision Matrix: When To Use Each Ingestion + Schema Approach

Requirement	Best Tool	Schema Config
One-time historical load	COPY INTO	No schema inference needed
Scheduled daily batch + fixed schema	COPY INTO	VALIDATE ALL, enforce schema
Continuous streaming + schema stable	Auto Loader (dir listing)	addNewColumns + rescue
Continuous + billions of files	Auto Loader (file events)	cloudFiles.useManagedFileEvents
Schema evolving over time	Auto Loader	rescue mode + mergeSchema
SaaS/ERP source (Salesforce, SAP)	Lakeflow Connect	Built-in schema handling

8.2 Exam Question Patterns

"new columns arrive..." → Think: schema evolution mode (addNewColumns, rescue, failOnNewColumns)

"_rescued_data column..." → Think: rescue mode or schema mismatch

"schema stored in schemaLocation..." → Think: required for inference

"type inference all STRING..." → Think: need cloudFiles.inferColumnTypes=true

"Delta table schema grows..." → Think: mergeSchema=true on writeStream

"DLT checkpoint managed..." → Think: only schemaLocation is required

QUIZ

9. Practice Questions (15 Questions)

Test Your Knowledge

Q1. Auto Loader samples up to 50 GB or 1000 files on first run to infer schema. What happens if you delete the schemaLocation directory while the stream is active?

- **A)** Stream continues unaffected; schema is persisted in memory
- **B)** Auto Loader re-infers schema from scratch on next trigger
- **C)** Stream stops permanently; requires manual restart with new schemaLocation
- **D)** Schema remains in the checkpoint directory as backup

Q2. You set cloudFiles.schemaEvolutionMode='rescue'. New column 'region' arrives. What is the expected behavior?

- **A)** Stream fails with UnknownFieldException until you restart
- **B)** Stream continues; 'region' is added to table schema automatically
- **C)** Stream continues; 'region' is placed in _rescued_data column
- **D)** Stream continues but silently drops 'region'

Q3. What is the difference between cloudFiles.schemaEvolutionMode='addNewColumns' and cloudFiles.schemaEvolutionMode='rescue'?

- **A)** addNewColumns stops the stream; rescue continues the stream
- **B)** They are the same; 'rescue' is an alias
- **C)** addNewColumns auto-updates table; rescue only captures in _rescued_data
- **D)** rescue is the default; addNewColumns must be set explicitly

Q4. You set mergeSchema='true' in writeStream. What does this option do?

- **A)** Enables cloudFiles.schemaEvolutionMode='merge'
- **B)** Allows Delta table schema to evolve to include new source columns
- **C)** Merges multiple Auto Loader streams into one checkpoint
- **D)** Same as cloudFiles.schemaHints

Q5. Which file formats are supported for Auto Loader schema inference?

- **A)** JSON, CSV, Parquet, ORC
- **B)** JSON, CSV, XML, Avro, Parquet
- **C)** Any format; inference is format-agnostic
- **D)** JSON and CSV only

Practice Questions (continued)

Q6. In DLT, which location must you explicitly specify for Auto Loader schema inference?

- **A)** checkpointLocation
- **B)** cloudFiles.schemaLocation
- **C)** Both A and B
- **D)** Neither; DLT manages both automatically

Q7. What is the default cloudFiles.schemaEvolutionMode when you provide an explicit schema?

- **A)** addNewColumns
- **B)** rescue
- **C)** none
- **D)** addNewColumnsWithTypeWidening

Q8. You want Auto Loader to widen INT columns to BIGINT automatically. What schemaEvolutionMode should you use?

- **A)** addNewColumns
- **B)** rescue
- **C)** addNewColumnsWithTypeWidening (DBR 16.4+)
- **D)** type widening is not supported

Q9. A source JSON file has 'Amount' (uppercase) but your schema has 'amount' (lowercase). Without special configuration, what happens?

- **A)** Auto Loader treats them as the same column; picks one case arbitrarily
- **B)** Column is rescued to _rescued_data due to case mismatch
- **C)** Stream fails with a case sensitivity error
- **D)** Column is silently dropped

Q10. You infer schema from JSON with cloudFiles.inferColumnTypes='false'. All columns become what type?

- **A)** Their natural JSON types (INT, STRING, BOOLEAN, etc.)
- **B)** STRING
- **C)** VARIANT
- **D)** It depends on the sampled data

ANS

10. Answer Key & Explanations

All 15 Questions Answered

Q1.

Answer: B

Deleting schemaLocation forces Auto Loader to re-infer the schema from scratch on the next trigger. This can cause type mismatches if source data has evolved.

Q2.

Answer: C

In rescue mode, the stream NEVER FAILS. New columns are saved as JSON in _rescued_data. You can inspect and gradually integrate them.

Q3.

Answer: A

addNewColumns (default) stops the stream when a new column is detected. rescue mode keeps the stream running and captures the new column in _rescued_data.

Q4.

Answer: B

mergeSchema is a writeStream OPTION (not a cloudFiles.schemaEvolutionMode value). It tells Delta Lake to add new source columns to the target table schema automatically.

Q5.

Answer: B

Supported: JSON, CSV, XML (DBR 14.3+), Avro (DBR 10.4+), Parquet (DBR 11.3+). Not supported: ORC, Text, binaryFile (these have fixed schemas).

Q6.

Answer: B

In DLT, you must set cloudFiles.schemaLocation. DLT automatically manages checkpointLocation, so you don't specify it.

Q7.

Answer: C

When you provide an explicit schema, the default mode is 'none' (new columns are ignored). If you DON'T provide a schema, the default is 'addNewColumns'.

Q8.

Answer: C

addNewColumnsWithTypeWidening (available in DBR 16.4+) automatically widens types like INT→BIGINT without manual intervention.

Q9.

Answer: A

Auto Loader treats abc, Abc, and ABC as the same column and arbitrarily picks one case. Use cloudFiles.schemaHints to enforce a specific case.

Q10.

Answer: B

With inferColumnTypes='false' (the default), JSON/CSV are all inferred as STRING. Set inferColumnTypes='true' to get actual types.

CHEAT

11. Master Cheat Sheet

Quick Reference

Schema Inference Options

- cloudFiles.schemaLocation ← REQUIRED for inference
- cloudFiles.inferColumnTypes → true for real types, false for all STRING
- cloudFiles.schemaHints → Override specific column types
- cloudFiles.schemaEvolutionMode → Control what happens with new columns

Schema Evolution Modes

- addNewColumns (default, no schema) → Stream fails, schema auto-updates, restart needed
- addNewColumnsWithTypeWidening (DBR 16.4+) → Auto-widens types (int→long)
- rescue → Stream never fails, new data goes to _rescued_data
- failOnNewColumns → Stream fails, schema NOT auto-updated, manual fix required
- none (default, schema provided) → New columns silently ignored

Rescued Data Column

- _rescued_data (auto-created) ← Captures: new columns, type mismatches, case mismatches
- rescuedDataColumn option → Rename _rescued_data to custom name
- Query: get_json_object(_rescued_data, '$.fieldname')

writeStream Options (NOT cloudFiles options)

- mergeSchema='true' → Delta table schema evolves with source
- checkpointLocation → REQUIRED, unique per stream
- outputMode='append' → Standard for ingestion

DLT Auto Loader Pattern

- @dlt.table + readStream.format('cloudFiles')
- Only set: cloudFiles.schemaLocation
- DLT manages: checkpointLocation, mergeSchema, restart

■ Exam Focus: Schema evolution modes are heavily tested. Know the 5 modes and when each fails vs. continues vs. rescues. Also, mergeSchema is a writeStream option, NOT a schemaEvolutionMode.

LAB

DEEP DIVE 1: Type Widening Lab

Auto Loader with Automatic Type Evolution

Scenario: e-commerce order system with evolving schema

Day 1: Orders ingested with order_id INT (max 2M), total DOUBLE

Day 30: Scale grows, order_id now exceeds INT range. New files have order_id BIGINT

Without Type Widening (addNewColumns mode):

■ Without Widening

```
# Day 30: New BIGINT column detected
# Result: Stream FAILS with UnknownFieldException
# Action: Manually update schema, restart stream
# Cost: Manual intervention, downtime
```

With Type Widening (addNewColumnsWithTypeWidening):

■ With Type Widening

```
# Configure with type widening
.option('cloudFiles.schemaEvolutionMode', 'addNewColumnsWithTypeWidening')

# Day 30: New BIGINT column detected
# Result: INT → BIGINT widening AUTOMATIC
# Stream FAILS (expected behavior), but on restart:
# → Schema updated with BIGINT
# → Stream resumes from checkpoint
# Cost: One auto-restart, no manual intervention
```

Supported Type Widening Chains:

Source Type	Widens To	Then To
BYTE	SHORT	INT → LONG → DECIMAL
SHORT	INT	LONG → DECIMAL
INT	LONG	DECIMAL
LONG	DECIMAL	N/A
FLOAT	DOUBLE	N/A
DECIMAL	DECIMAL (wider)	N/A

■■ **If widening is unsupported (e.g., INT → STRING), the value is placed in _rescued_data. Always set mergeSchema=true to update target table.**

LAB

DEEP DIVE 2: Case Sensitivity & Column Matching

Handling Mixed-Case Column Names

Scenario: Data from multiple teams with inconsistent casing

■ Mixed-Case Source Data

```
# Day 1: Team A sends JSON with columns
{'event_id': '123', 'user_name': 'alice'}

# Day 2: Team B sends JSON with columns
{'Event_Id': '456', 'User_Name': 'bob'}

# Day 3: Team C sends JSON with columns
{'EVENT_ID': '789', 'USER_NAME': 'charlie'}
```

Default Behavior (Case-Insensitive):

Auto Loader treats event_id, Event_Id, and EVENT_ID as the **same column**. It arbitrarily picks **one case** (e.g., event_id) based on first sampled data. Other variations become case mismatches and are rescued to _rescued_data.

■ Default Rescuing Behavior

```
-- Result table might have:
event_id | _rescued_data
---------|--------------------
'123'    | NULL  -- matches inferred case
'456'    | {"Event_Id": "456"}  -- case mismatch rescued
'789'    | {"EVENT_ID": "789"}  -- case mismatch rescued
```

Solution 1: Use Schema Hints to Enforce Case

■ Enforce Specific Case

```
.option('cloudFiles.schemaHints',
  'event_id INT, user_name STRING')
```

Solution 2: Set readerCaseSensitive=false

■ Case-Insensitive Reading

```
.option('readerCaseSensitive', 'false')
# Treats all case variations as the same column
```

Solution 3: Standardize Upstream (Best Practice)

Require all upstream teams to use lowercase_snake_case column names before ingestion. This avoids all case-sensitivity issues and improves data quality.

LAB

DEEP DIVE 3: Rescue Mode End-to-End Workflow

Detect, Monitor, Integrate New Columns

Complete process for safe schema evolution in production

Step 1: Configure with Rescue Mode

■ **Production Setup**

```
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaEvolutionMode', 'rescue')
  .option('cloudFiles.schemaLocation', '/schema')
  .load('/data/')

(df.writeStream
  .format('delta')
  .option('mergeSchema', 'true')
  .option('checkpointLocation', '/ckpt')
  .table('bronze.events'))
```

Step 2: Monitor for Rescued Data

■ **Monitoring Query**

```
-- Daily check for rescued data
SELECT
  DATE(current_date()) as check_date,
  COUNT(*) as rescued_records,
  COUNT(DISTINCT _rescued_data) as unique_patterns
FROM bronze.events
WHERE _rescued_data IS NOT NULL
  AND DATE(_ingest_date) = CURRENT_DATE();
```

Step 3: Extract & Inspect New Fields

■ **Field Extraction**

```
-- Extract newly arrived field 'region'
SELECT
  event_id,
  get_json_object(_rescued_data, '$.region') as region
FROM bronze.events
WHERE _rescued_data IS NOT NULL
  AND get_json_object(_rescued_data, '$.region') IS NOT NULL
LIMIT 10;
```

Step 4: Add Column to Schema & Update writeStream

■ **Schema Update**

```
-- Add column to table
ALTER TABLE bronze.events ADD COLUMN region STRING;

-- Update writeStream with mergeSchema=true
# Stream will auto-merge 'region' into table on next run
```

Step 5: Archive Rescue Column

■ **Cleanup**

```
-- Once integrated, set _rescued_data to NULL
UPDATE bronze.events
SET _rescued_data = NULL
WHERE get_json_object(_rescued_data, '$.region') IS NOT NULL;
```

LAB

DEEP DIVE 4: Parquet Schema Merging

How Auto Loader Handles Multi-File Type Evolution

Parquet files encode schema in file metadata. When Auto Loader ingests multiple Parquet files with different schemas, it must merge them intelligently.

Scenario: Same table, evolving over time

■ Schema Evolution Across Files

```
# File 1 (Day 1): Parquet schema
order_id    INT
customer_id INT
amount      DOUBLE

# File 2 (Day 2): Parquet schema (new column)
order_id    INT
customer_id INT
amount      DOUBLE
currency    STRING  ← NEW

# File 3 (Day 3): Parquet schema (type change)
order_id    LONG          ← Changed from INT
customer_id INT
amount      DOUBLE
currency    STRING
region      STRING        ← NEW
```

Auto Loader's Merging Strategy:

- 1. Samples all files to detect schema variations
- 2. For new columns → adds to merged schema
- 3. For type mismatches → selects **widest type**
- 4. Final merged schema = union of all file schemas with widest types

Merged Schema Result:

■ Final Inferred Schema

```
order_id    LONG       ← Widest (was INT, became LONG)
customer_id INT        ← Unchanged
amount      DOUBLE     ← Unchanged
currency    STRING     ← Added
region      STRING     ← Added
```

■■ **Important: Use cloudFiles.schemaHints if you disagree with Auto Loader's type selection (e.g., you want DECIMAL instead of DOUBLE).**

EXAM

DEEP DIVE 5: Common Pitfalls & Exam Traps

What Gets Wrong on Tests

Pit fall 1: Confusing schemaEvolutionMode with mergeSchema

■ .option('cloudFiles.schemaEvolutionMode', 'merge') — WRONG, no such mode ■ .option('mergeSchema', 'true') — Correct writeStream option

Pitfall 2: Assuming rescue mode will auto-add columns to table

rescue mode ONLY saves to _rescued_data, it does NOT update the table schema. Must also set mergeSchema=true in writeStream.

Pitfall 3: Forgetting that DLT manages checkpointLocation

■ In DLT, do NOT set checkpointLocation ■ Only set cloudFiles.schemaLocation if you want inference

Pitfall 4: Not setting cloudFiles.schemaLocation for inference

Without schemaLocation, schema inference is DISABLED. Result: No schema stored, stream cannot restart reliably.

Pitfall 5: Mixing CSV with expectation of auto-type inference

CSV defaults to all STRING. Must set cloudFiles.inferColumnTypes=true JSON & Parquet: types inferred by default.

Pitfall 6: Deleting schemaLocation and expecting stream to continue

Stream will re-infer schema from scratch → potential type mismatches.

Pitfall 7: Using cloudFiles.useNotifications instead of useManagedFileEvents

useNotifications = classic file notification (deprecated style) useManagedFileEvents = modern file events (recommended for new projects)

FINAL

Final Comprehensive Cheat Sheet

All Key Concepts
One Page

Schema Inference Configuration

- cloudFiles.schemaLocation: REQUIRED for inference. Stores inferred schema.
- cloudFiles.inferColumnTypes: false (default) = all STRING; true = actual types
- cloudFiles.schemaHints: Override specific column types (e.g., 'amount DECIMAL(10,2)')
- rescuedDataColumn: Rename _rescued_data to custom column name

Schema Evolution Modes (cloudFiles.schemaEvolutionMode)

addNewColumns: Default (no schema), Stream fails on new col, auto-updates schema, restart to resume

addNewColumnsWithTypeWidening: Like addNewColumns + auto-widens types (DBR 16.4+)

rescue: Stream never fails, new data → _rescued_data, no schema evolution

failOnNewColumns: Stream fails, schema NOT auto-updated, requires manual fix

none: Default (schema provided), new cols silently ignored

writeStream Options (NOT cloudFiles options)

- mergeSchema=true: Allow Delta table to evolve with source schema
- checkpointLocation: REQUIRED, unique per stream
- outputMode='append': Standard for ingestion

DLT Best Practice

✓ Set cloudFiles.schemaLocation
✗ Do NOT set checkpointLocation (DLT manages it)
✓ DLT auto-handles mergeSchema and restarts

SCENARIO 1: Financial Transaction Pipeline

Schema Evolution in Production Banking System

Requirements: Ingest transactions from multiple regional branches, each sending slightly different JSON structure. Must never lose data. Regulatory compliance requires exact schema tracking.

Architecture Decision:

■ Multi-Region Financial Data

```
# US Region sends: {transaction_id, amount_usd, timestamp}
# UK Region sends: {transaction_id, amount_gbp, timestamp, fx_rate}
# EU Region sends: {transaction_id, amount_eur, timestamp, fx_rate, bank_code}

# Solution: Rescue mode + strict monitoring
.option('cloudFiles.schemaEvolutionMode', 'rescue')
.option('cloudFiles.schemaLocation', '/schema')
.option('mergeSchema', 'true')

# Stream NEVER fails, all data captured
# Each region's new fields → _rescued_data initially
# Then manually validated and integrated one-by-one
```

Daily Monitoring Query:

■ Alert Monitoring

```
-- Alert if rescue data volume exceeds threshold (indicating new region)
SELECT
  DATE(current_date()) as date,
  COUNT(*) as rescued_records,
  get_json_object(MAX(_rescued_data), '$.bank_code') as new_field_sample
FROM transactions
WHERE _rescued_data IS NOT NULL
GROUP BY DATE(current_date())
HAVING COUNT(*) > 100  -- Alert threshold
```

SCENARIO 2: IoT Sensor Data Ingestion

Handling Device Firmware Updates That Change Schema

Context: 10,000 sensors worldwide, each sending temperature + humidity readings. New sensor model (v2) rolls out gradually over 6 months with different metrics. Cannot wait for all devices to upgrade.

Schema Evolution Timeline:

Month	Sensor Mix	Schema	Auto Loader Mode
1	100% v1	sensor_id, temp, humidity	rescue (safe default)
2-3	80% v1, 20% v2	v2 has battery_pct → rescue	rescue (continues)
4-5	50% v1, 50% v2	v2 also has signal_strength → rescue	rescue (continues)
6	100% v2	All fields present, integrate	Manually add columns

■ Gradual Rollout Handling

```
# Configuration handles entire roll-out without downtime
```

```
.option('cloudFiles.schemaEvolutionMode', 'rescue')
.option('mergeSchema', 'true')

# As v2 devices increase, new fields appear in _rescued_data
# Monitor ratio of rescued vs. normal records
# When 90% of data has battery_pct (e.g., Month 5-6),
# add column to table and remove from rescue
```

DEBUG

Troubleshooting Guide

Common Errors & Solutions

■ AnalysisException: schemaLocation required

Cause: Attempted schema inference without setting cloudFiles.schemaLocation Fix: Add .option('cloudFiles.schemaLocation', '/path/to/schema')

■ UnknownFieldException: Column XYZ not in schema

Cause: New column detected, using addNewColumns mode (default) Fix: Restart stream (it auto-updated schema)

■ Stream does not fail when new columns arrive (but was expecting it)

Cause: Using rescue or none mode Fix: Change to .option('cloudFiles.schemaEvolutionMode', 'addNewColumns') if you want failure

■ Data not appearing in _rescued_data despite new columns

Cause: schemaEvolutionMode=none (default when explicit schema provided) Fix: Set schemaEvolutionMode='rescue' or provide rescuedDataColumn option

■ Type mismatch: Column amount is DOUBLE but got STRING value

Cause: CSV inference all-STRING, but DOUBLE expected Fix: .option('cloudFiles.inferColumnTypes', 'true')

■ Schema constantly re-inferred on each run (no consistency)

Cause: schemaLocation deleted or changed between runs Fix: Keep schemaLocation stable, never delete while stream active

■ Delta table schema doesn't grow even though new columns in source

Cause: mergeSchema not set in writeStream Fix: Add .option('mergeSchema', 'true') to writeStream

ADVANCED

Schema Hints: Advanced Patterns

Complex Types & Nested Structures

Pattern 1: Complex Nested Structures

Nested Field Hints

```
# Source has nested objects
{
  'order_id': '123',
  'customer': {
    'id': '456',
    'addresses': [
      {'street': '123 Main', 'zip': '90210'}
    ]
  }
}

.option('cloudFiles.schemaHints',
  'customer.id INT,
   customer.addresses.element.zip STRING')
```

Pattern 2: Map Types

Map Type Hints

```
# Source has map/dictionary fields
{
  'event_id': '789',
  'properties': {
    'color': 'red',
    'size': 'large'
  }
}

.option('cloudFiles.schemaHints',
  'properties MAP<STRING,STRING>')
```

Pattern 3: Array Types

Array Type Hints

```
# Source has array fields
# Inferred: tags ARRAY<STRING>
# But want: tags ARRAY<INT>

.option('cloudFiles.schemaHints',
  'tags ARRAY<INT>')
```

DLT

DLT Expectations & Data Quality

Using Expectations with Schema Evolution

DLT with Expectations

```
import dlt

@dlt.table(
  name='bronze_orders',
  comment='Orders via Auto Loader'
)
@dlt.expect_or_drop('valid_order_id', 'order_id IS NOT NULL')
@dlt.expect_or_drop('positive_amount', 'amount > 0')
def bronze_orders():
  return (
    spark.readStream.format('cloudFiles')
      .option('cloudFiles.format', 'json')
      .option('cloudFiles.schemaEvolutionMode', 'rescue')
      .option('cloudFiles.schemaLocation',
        '/Volumes/main/bronze/schema/orders')
      .load('s3://bucket/orders/')
  )

# DLT ensures data quality + schema evolution
# Expectations: reject invalid records
# Schema evolution: capture unexpected fields in _rescued_data
```

Best Practice: Expectations + Rescue

Use **expectations for data validation** (business logic) and **rescue mode for schema flexibility** (structural changes). This combination ensures both data quality and schema resilience.

EXAM

Exam Question Patterns

What Test Writers Focus On

Q: "Stream failed with UnknownFieldException..."

A: → addNewColumns mode (default). Schema auto-updated. Restart needed.

Q: "Stream never fails even with new columns..."

A: → rescue or none mode. Check if data is in _rescued_data.

Q: "Table schema didn't grow..."

A: → mergeSchema=false (default). Set to true or manually ALTER TABLE.

Q: "_rescued_data is NULL..."

A: → Mode is 'none' (default w/schema) or all incoming data matches schema.

Q: "Case sensitivity: abc, Abc, ABC..."

A: → Auto Loader picks one case arbitrarily. Use schemaHints to enforce.

Q: "ORC files ingestion..."

A: → ORC not supported for schema inference. Use Parquet instead.

Q: "DLT checkpoint managed automatically..."

A: → TRUE. Only set cloudFiles.schemaLocation.

Q: "Type widening INT to BIGINT..."

A: → addNewColumnsWithTypeWidening mode (DBR 16.4+).

Q: "Schema inference takes 50 GB or 1000 files..."

A: → Correct. First run samples up to 50GB or 1000 files, whichever first.

REF

Quick Reference Card

Print This Page

REQUIRED Options (for inference)

✓ cloudFiles.format
✓ cloudFiles.schemaLocation
✓ checkpointLocation (writeStream)

Default Behaviors

cloudFiles.inferColumnTypes = false (all STRING)
cloudFiles.schemaEvolutionMode = addNewColumns (no schema), none (schema provided)
mergeSchema = false (table schema doesn't grow)
outputMode = append (standard)

DO NOT Confuse

■■ schemaEvolutionMode ≠ mergeSchema (former is cloudFiles, latter is writeStream)
■■ cloudFiles.useNotifications ≠ cloudFiles.useManagedFileEvents
■■ _rescued_data (column) ≠ rescuedDataColumn (option)

Summary & Key Takeaways

✓ Schema Inference automatically samples first 50GB or 1000 files

✓ cloudFiles.schemaLocation is REQUIRED for inference and must be stable

✓ Five schema evolution modes exist: addNewColumns (default), addNewColumnsWithTypeWidening, rescue, failOnNewColumns, none

✓ rescue mode is ideal for production—stream never fails, all data captured in _rescued_data

✓ mergeSchema is a writeStream OPTION, not a schemaEvolutionMode value

✓ DLT auto-manages checkpointLocation—only set cloudFiles.schemaLocation

✓ Type widening (INT→BIGINT) requires addNewColumnsWithTypeWidening mode (DBR 16.4+)

✓ Schema hints override inferred types without requiring full explicit schema

✓ cloudFiles.inferColumnTypes=false (default) means JSON/CSV inferred as all STRING

✓ ORC is NOT supported for schema inference—use Parquet instead

Week 4 Section 4: Exam Preparation Checklist

■ Understand the difference between schema inference and schema evolution

■ Know all 5 schemaEvolutionMode values and when each is used

■ Understand _rescued_data column: what triggers it, how to query it

■ Know mergeSchema is writeStream, NOT cloudFiles option

■ Understand default modes change based on whether schema is provided

■ Know cloudFiles.schemaLocation is required and must be stable

■ Understand DLT manages checkpoint, you only set schemaLocation

■ Know type widening modes and which runtime version supports it

■ Understand schema hints syntax for nested and complex types

■ Know case sensitivity behavior and solutions (hints, readerCaseSensitive)

■ Understand rescue mode is ideal production pattern

■ Know CSV defaults to all STRING, use inferColumnTypes for real types

■ Know ORC is unsupported for inference

■ Understand file events (useManagedFileEvents) vs classic notifications

■ Practice writing recovery patterns: monitor rescue data → integrate new fields

Official Databricks Documentation Links

■ Core Auto Loader Documentation:

- Configure schema inference and evolution
  `https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/schema`
- Auto Loader options reference
  `https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/options.html`
- Auto Loader with file events
  `https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/file-events-explained`
- Configure file notification mode
  `https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/file-notification-mode`
- Automatic type widening
  `https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/type-widening`
- Auto Loader production best practices
  `https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/production`

■ Related Topics (From Week 3 & 4):

- Spark API options reference (including cloudFiles)
- Data Ingestion overview
- Delta Live Tables documentation

Study Strategy for Exam: Focus heavily on schema evolution modes. Practice writing rescue mode pipelines and monitoring rescued data. Understand the difference between schemaEvolutionMode and mergeSchema. Know which options are required, which are optional, and what the defaults are.